

LABORATORY

LAB 11

Motion Data Monitoring with MPU6050

Reading Acceleration and Angular Velocity Using the MPU6050
Device API with EduFramework

S32K144 · MaaZEDU

Contents

1	Introduction	2
1.1	Lab Overview	2
1.2	Objectives	2
2	Background	2
2.1	Accelerometer and Gyroscope	2
2.2	I2C Communication	3
3	Hardware Setup	3
3.1	Required Hardware	3
3.2	I2C Signal Mapping	3
3.3	Connecting the MPU6050	3
4	EduFramework API	4
4.1	MPU_Data_t Structure	4
4.2	MPU_Begin()	5
4.3	MPU_ReadData()	5
5	Lab Exercise	6
5.1	Requirements	6
5.2	Program	6
5.3	Program Explanation	7
5.4	Verification	8
6	Extension	8
6.1	Selective Reading and Status Checking	8
6.2	Calibration with MPU_Calibrate()	9
6.3	I2C Address and the AD0 Pin	9
6.4	Customizing Measurement Ranges	10
6.5	Context-Based Advanced API	10
6.6	Extension Exercise	11
7	References	11

1 Introduction

1.1 Lab Overview

The MPU6050 is a motion sensor that integrates a three-axis accelerometer and a three-axis gyroscope. These two sensing elements provide measurements along three mutually perpendicular axes, allowing an embedded system to observe changes in motion and the rotational rate of the object to which the sensor is attached [1]. This lab focuses on using the MPU6050 through the EduFramework Device API rather than accessing the sensor registers directly. The program initializes the module using the default configuration, simultaneously reads acceleration, angular velocity, and sensor temperature data, and then displays the results on the Serial Monitor. The Extension section introduces selective-read APIs, calibration, and the advanced interface that allows the I2C address as well as the accelerometer and gyroscope measurement ranges to be changed [6].

1.2 Objectives

OBJECTIVES

The objectives of this laboratory are:

1. Describe the roles of the accelerometer and gyroscope in a six-axis motion sensor.
2. Explain the basic components of I2C communication, including `SDA`, `SCL`, and device addressing.
3. Use the `MPU_Data_t` structure to access acceleration, angular velocity, and sensor temperature data provided by EduFramework.
4. Use `MPU_Begin()` and `MPU_ReadData()` to build an MPU6050 data-monitoring application on the Serial Monitor.
5. Recognize the advanced configuration capabilities of the MPU6050 Device API, including calibration, I2C address selection, and measurement-range configuration.

2 Background

2.1 Accelerometer and Gyroscope

A typical IMU uses three accelerometers and three rate gyroscopes arranged along three mutually perpendicular axes. The gyroscope provides the components of the object's angular velocity along the sensor axes, while the accelerometer provides measurements of specific force along the corresponding axes [1].

In EduFramework, the MPU6050 accelerometer values are converted to units of `g`, while the gyroscope data are converted to `dps` (degrees per second). When the module is stationary, the accelerometer data still reflect the effect of gravity, so one axis may have a magnitude close to `1 g` depending on the sensor orientation. Gyroscope data while stationary are typically close to `0 dps`, but offset and noise are still present [1] [6].

Note

A gyroscope provides angular velocity, not the rotation angle directly. Deriving orientation, pitch, roll, or yaw from IMU data requires additional processing and is outside the scope of this lab.

2.2 I2C Communication

I2C is a synchronous two-wire serial bus. `SCL` carries the clock signal, while `SDA` is used to transfer data. Multiple devices can share the same bus and are distinguished by the address assigned to each peripheral [2]. The MPU6050 uses I2C as the host interface in this lab configuration. The least significant address bit is determined by the `AD0` pin: when `AD0` is low, the slave address is `0x68`; when `AD0` is high, the address becomes `0x69` [3].

The data flow in this lab can be represented as:

MPU6050 → I2C → Wire / LPI2C → MPU6050 Device API → Application

3 Hardware Setup

3.1 Required Hardware

The lab uses the MaaZEDU Development Board and an MPU6050 module. MaaZEDU is a development board based on the S32K144 microcontroller [5].

The module is powered from `3.3V`, shares `GND` with the board, and exchanges data through the I2C interface [3].

Hardware	Description
MaaZEDU Development Board	Development board based on the S32K144 microcontroller.
MPU6050 module	Sensor module integrating a three-axis accelerometer and a three-axis gyroscope.
Jumper wires	Connect power and I2C signals between the MPU6050 and MaaZEDU.
USB Cable	Connect the board to the computer for power, program upload, and Serial Monitor access.

Table 11.3.1. Hardware used in the lab exercise

3.2 I2C Signal Mapping

In the current EduFramework configuration, the I2C0 interface uses the `I2C0_SCL` and `I2C0_SDA` signals. These signals are used by the Wire API and by the MPU6050 Device API in this lab exercise [6].

Component	Logical Pin	MCU Pin	Function
MPU6050 <code>SCL</code>	<code>I2C0_SCL</code>	<code>PTA3</code>	I2C Clock
MPU6050 <code>SDA</code>	<code>I2C0_SDA</code>	<code>PTA2</code>	I2C Data

Table 11.3.2. MPU6050 signal mapping used in the lab exercise

3.3 Connecting the MPU6050

In the default lab configuration, `VCC` is connected to `3.3V`, `GND` to `GND`, `SCL` to `I2C0_SCL`, and `SDA` to `I2C0_SDA`. The `AD0` pin is held low by connecting it to `GND`, so the Device uses the default address `0x68` [3] [6].

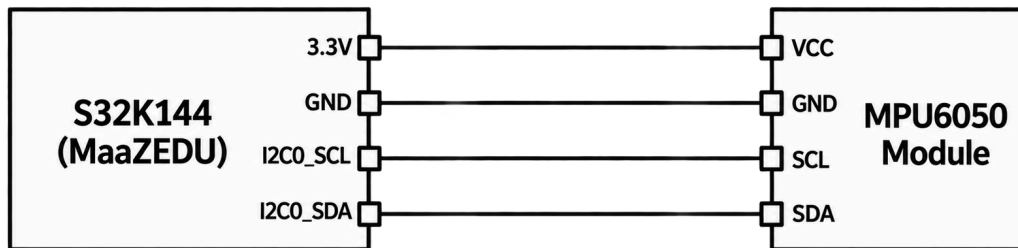


Figure 11.3.1. MPU6050 connection diagram with MaaZEDU Development Board

Note

The `INT` pin is not used in this lab exercise. Data are read actively through `MPU_ReadData()` according to the program cycle.

4 EduFramework API

The EduFramework MPU6050 Device provides a group of simple APIs for basic applications and a group of advanced APIs for cases that require detailed configuration. The main lab exercise uses `MPU_Data_t`, `MPU_Begin()`, and `MPU_ReadData()`; the advanced configuration APIs are introduced in the Extension section [6].

4.1 MPU_Data_t Structure

`MPU_Data_t` groups the values from one sensor read into a single structure. The application can access each field directly after `MPU_ReadData()` completes successfully [6].

```
typedef struct
{
    float accelX;
    float accelY;
    float accelZ;
    float gyroX;
    float gyroY;
    float gyroZ;
    float temperature;
} MPU_Data_t;
```

Listing 11.4.1. MPU_Data_t data structure

Field	Quantity	Unit
accelX	Acceleration / specific force along the X-axis	g
accelY	Acceleration / specific force along the Y-axis	g
accelZ	Acceleration / specific force along the Z-axis	g
gyroX	Angular velocity along the X-axis	dps
gyroY	Angular velocity along the Y-axis	dps
gyroZ	Angular velocity along the Z-axis	dps
temperature	Internal MPU6050 sensor temperature	°C

Note

The `temperature` field reflects the internal temperature of the MPU6050 sensor. This value should not be used as a direct measurement of the surrounding ambient temperature [3].

4.2 MPU_Begin()

`MPU_Begin()` initializes the MPU6050 using the default EduFramework configuration. The API initializes the Wire layer, uses address `0x68`, wakes the sensor, and configures the accelerometer range to ± 2 g and the gyroscope range to ± 250 dps [6].

MPU_Begin

Initialize the MPU6050 using the default EduFramework configuration.

Syntax

```
MPU_Begin();
```

Return value

`true` when initialization and communication verification succeed; `false` when the sensor cannot be communicated with or configured.

Example:

```
if (true == MPU_Begin())
{
    /* MPU6050 is ready. */
}
```

4.3 MPU_ReadData()

`MPU_ReadData()` performs one MPU6050 measurement-frame read and simultaneously updates the acceleration, gyroscope, and temperature fields in `MPU_Data_t`. The API returns data only when the Device has been initialized successfully [6].

MPU_ReadData

Read one measurement frame and update the supplied data structure.

Syntax

```
MPU_ReadData(data);
```

Parameters

Parameter	Type / accepted value	Description
data	MPU data structure	Storage for acceleration, gyroscope, and temperature data after the read.

Return value

`true` when the data are read successfully; `false` when the Device has not been initialized, the parameter is invalid, or I2C communication fails.

Example:

```
MPU_Data_t data;

if (true == MPU_ReadData(&data))
{
    Serial1_printFloat(data.accelX);
}
```

5 Lab Exercise

5.1 Requirements

Build a program that monitors motion data from the MPU6050 and displays the results on the Serial Monitor. The program uses `MPU_Begin()` to initialize the Device, then calls `MPU_ReadData()` every `1000 ms` to obtain three-axis acceleration, three-axis gyroscope, and sensor-temperature data.

The Serial Monitor uses a baud rate of `9600`. If initialization fails, the program must report the error and must not continue reading the sensor. If an individual read fails, the program prints an error message for that read and retries on the next cycle.

5.2 Program

In `src/main.c`, implement the hardware-verified program as follows:

```
#include "Arduino.h"
#include "MPU6050.h"
int main(void)
{
    MPU_Data_t data;
    setup();
    Serial1_begin(9600U);
    Serial1_println("MPU6050 Sensor Monitor");
    Serial1_println("-----");
    if (false == MPU_Begin())
    {
        Serial1_println("MPU6050 initialization failed.");
        while (1)
        {
            delay(1000U);
        }
    }
    Serial1_println("MPU6050 initialized.");
    while (1)
    {
        if (true == MPU_ReadData(&data))
        {
            Serial1_print("Accel X: ");
            Serial1_printFloat(data.accelX);
            Serial1_print(" g | Y: ");
            Serial1_printFloat(data.accelY);
            Serial1_print(" g | Z: ");
            Serial1_printFloat(data.accelZ);
            Serial1_println(" g");
            Serial1_print("Gyro X: ");
            Serial1_printFloat(data.gyroX);
            Serial1_print(" dps | Y: ");
            Serial1_printFloat(data.gyroY);
            Serial1_print(" dps | Z: ");
            Serial1_printFloat(data.gyroZ);
            Serial1_println(" dps");
            Serial1_print("Temperature: ");
            Serial1_printFloat(data.temperature);
            Serial1_println(" C");
            Serial1_println("-----");
        }
        else
        {
            Serial1_println("MPU6050 read failed.");
        }
        delay(1000U);
    }
}
```

Listing 11.5.1. MPU6050 data-monitoring program

5.3 Program Explanation

`setup()` initializes the underlying EduFramework components, and `Serial1_begin(9600U)` prepares the Serial Monitor channel. `MPU_Begin()` then performs the complete initialization required for the default

MPU6050 configuration. If the API returns `false`, the program remains in the error loop to avoid continuing with a Device that is not ready [6].

Inside the main loop, `MPU_ReadData(&data)` reads one measurement frame. When the read is successful, all data have already been converted and stored in `data`, so the application only needs to access the `accelX`, `accelY`, `accelZ`, `gyroX`, `gyroY`, `gyroZ`, and `temperature` fields. The results are sent to the Serial Monitor using the Serial APIs introduced in previous labs [6].

The application flow can be summarized as:

Motion → MPU6050 → I2C → MPU_ReadData() → MPU_Data_t → Serial1 → Serial Monitor
`delay(1000U)` creates a one-second interval between updates, making the Serial Monitor output easier to observe.

5.4 Verification

Build and upload the program to the MaaZEDU Development Board, then open the Serial Monitor at a baud rate of `9600`.

First, place the module at rest on a stable surface. When one sensor axis is nearly aligned with gravity, the acceleration component on that axis has a magnitude close to `1 g`, while the gyroscope values on all three axes remain close to `0 dps`. Actual values may differ slightly because of module orientation, sensor offset, and noise [1].

Next, tilt the module to observe how acceleration is distributed among the axes, then rotate the module to observe changes in the gyroscope values. The values are not required to match a fixed set of numbers; the objective is to verify that the measurements respond appropriately to physical motion.

Data observed during hardware testing have the following form:

```
MPU6050 Sensor Monitor
-----
MPU6050 initialized.
Accel X: 0.060 g | Y: 0.005 g | Z: 1.030 g
Gyro X: 1.901 dps | Y: -1.534 dps | Z: -0.191 dps
Temperature: 46.412 C
-----
```

EXPECTED RESULT

The Serial Monitor continuously displays three-axis acceleration and gyroscope data together with the sensor temperature. When the module is tilted or rotated, the corresponding values change with the motion; when the module is stationary, the gyroscope values remain near `0 dps`, while the acceleration values reflect the gravity component according to the sensor orientation.

6 Extension

The main lab exercise uses the Beginner API to keep the configuration and program flow simple. The MPU6050 Device also provides selective-read APIs, calibration, and a context-based Advanced API so that an application can choose the I2C address, measurement ranges, and number of calibration samples when required [6].

6.1 Selective Reading and Status Checking

In addition to `MPU_ReadData()`, the Beginner API provides functions for reading individual groups of data:

API	Function
<code>MPU_IsInitialized()</code>	Check the initialization state of the default Device.
<code>MPU_ReadAcceleration()</code>	Read acceleration along the X, Y, and Z axes.
<code>MPU_ReadGyroscope()</code>	Read gyroscope data along the X, Y, and Z axes.
<code>MPU_ReadTemperature()</code>	Read the internal MPU6050 sensor temperature.

Each selective-read API performs a new sensor read before returning the result. Therefore, when an application needs acceleration, gyroscope, and temperature data at the same time, `MPU_ReadData()` is more appropriate because the values are obtained from the same measurement frame and require only one Device read cycle [6]. Example when only acceleration is required:

```
float x = 0.0F;
float y = 0.0F;
float z = 0.0F;

if (true == MPU_ReadAcceleration(&x, &y, &z))
{
    /* Use acceleration data. */
}
```

6.2 Calibration with `MPU_Calibrate()`

`MPU_Calibrate()` calibrates the default MPU6050 by collecting multiple samples while the sensor is stationary and calculating accelerometer and gyroscope offsets. In the current implementation, the Beginner API uses 500 samples. The algorithm assumes that the module is held steady with the +Z axis close to +1 g during calibration [6].

`MPU_Calibrate`

Calibrate the default MPU6050 and update the offsets applied to subsequent readings.

Syntax

```
MPU_Calibrate();
```

Return value

`true` when the complete calibration process finishes successfully; `false` when the Device has not been initialized or an error occurs while collecting samples.

Note

Keep the module stationary and approximately level throughout calibration. Moving the sensor while samples are being collected causes the resulting offsets to no longer represent the intended static condition.

6.3 I2C Address and the `AD0` Pin

The default lab configuration connects `AD0` to `GND`, corresponding to address `0x68`. The MPU6050 also allows `AD0` to be driven to a logic-high level to use address `0x69`. This mechanism allows two MPU6050 devices to coexist on one I2C bus when one Device uses address `0x68` and the other uses `0x69` [3]. EduFramework defines two address constants:

```
MPU6050_ADDRESS_DEFAULT /* 0x68 */
MPU6050_ADDRESS_ALT     /* 0x69 */
```

The Beginner API always uses the default address. When address selection is required, the application uses the Advanced API and a separate `MPU6050_t` context. With the Advanced API, the Wire layer must be initialized before `MPU6050_begin()` is called [6].

Example:

```
MPU6050_t mpu;
Wire_begin();
if (true == MPU6050_begin(&mpu, MPU6050_ADDRESS_ALT))
{
    /* Device at address 0x69 is ready. */
}
```

Listing 11.6.1. Initializing the MPU6050 with a custom I2C address

6.4 Customizing Measurement Ranges

The MPU6050 Product Specification supports four accelerometer full-scale ranges and four gyroscope full-scale ranges. EduFramework exposes these choices through the Advanced API [3] [6].

Accelerometer	Gyroscope
MPU6050_RANGE_2_G	MPU6050_RANGE_250_DEG
MPU6050_RANGE_4_G	MPU6050_RANGE_500_DEG
MPU6050_RANGE_8_G	MPU6050_RANGE_1000_DEG
MPU6050_RANGE_16_G	MPU6050_RANGE_2000_DEG

The corresponding configuration APIs are:

```
MPU6050_setAccelerometerRange(&mpu, MPU6050_RANGE_4_G);
MPU6050_setGyroRange(&mpu, MPU6050_RANGE_500_DEG);
```

A larger measurement range allows the sensor to represent motion with a larger amplitude, while the sensitivity in counts per unit changes with the selected range. The measurement range should therefore be selected according to the expected motion amplitude of the application rather than always using the largest range [3].

6.5 Context-Based Advanced API

The Advanced API stores the address, measurement ranges, measurement data, and calibration offsets in `MPU6050_t`. This model allows the application to control the Device configuration instead of relying on the default context used by the Beginner API [6].

The main APIs are:

Advanced API	Role
<code>MPU6050_begin()</code>	Initialize a context using the selected I2C address.
<code>MPU6050_setAccelerometerRange()</code>	Change the accelerometer measurement range.
<code>MPU6050_setGyroRange()</code>	Change the gyroscope measurement range.
<code>MPU6050_read()</code>	Read a measurement frame into the context.
<code>MPU6050_calibrate()</code>	Calibrate using a sample count selected by the application.
<code>MPU6050_getAccelerationX()</code> , <code>MPU6050_getAccelerationY()</code> , <code>MPU6050_getAccelerationZ()</code>	Read the stored acceleration value for each axis from the context.
<code>MPU6050_getGyroX()</code> , <code>MPU6050_getGyroY()</code> , <code>MPU6050_getGyroZ()</code>	Read the stored gyroscope value for each axis from the context.
<code>MPU6050_getTemperature()</code>	Read the stored temperature from the context.

The Beginner API is suitable when only one MPU6050 with the standard configuration is required. The Advanced API is appropriate when the application requires a different address, a different measurement range, a custom calibration sample count, or explicit management of the Device context.

6.6 Extension Exercise

Configure the MPU6050 at address `0x69` by setting `AD0` to a logic-high level and use the Advanced API to:

- initialize a separate `MPU6050_t` ;
- select an accelerometer range of ± 4 g ;
- select a gyroscope range of ± 500 dps ;
- perform calibration using a sample count selected by the program;
- read the data and display acceleration and gyroscope values on the Serial Monitor.

Do not use `MPU_Begin()` in this exercise. The objective is to work directly with `MPU6050_begin()` , the range-configuration APIs, and `MPU6050_calibrate()` .

EXPECTED RESULT

The MPU6050 is initialized successfully at address `0x69` , data continue to be read and displayed after the measurement ranges are changed, and the stationary gyroscope values improve after an appropriate calibration process.

7 References

- [1] University of California, Berkeley, *Strapdown inertial navigation* , Rotations.
- [2] Cornell University, *Inter-Integrated Circuit (I2C)* , ECE 4760 - Designing with Microcontrollers.
- [3] InvenSense Inc., *MPU-6000 and MPU-6050 Product Specification* , PS-MPU-6000A-00, Rev. 3.4, 2013.
- [4] NXP Semiconductors, *S32K1xx MCU Family - Data Sheet* , S32K1XX, Rev. 15, 2026.
- [5] MaaZEDU, *MaaZEDU Development Board Guide* , Rev. 1.0, 2025.
- [6] EduFramework, *MPU6050 Device API* , EduFramework Source Code.